Deliverable 2 Report
EECS 3311Z - Software Design

https://github.com/egeyesss/eecs3311-reservation

Student Name - Student Number - Github Name

Ege Yesilyurt - 219701739 - egeyesss

Adren Reuel Singh - 219191881- adrenreuel

Weiqin Situ - 219720432 - kevinsitu1706

Gurnoor Kahlon - 220596086 - gurnoor-kahlon

Marwan Al-Jaufy - 219253459 - maljaufy

Mar 22, 2026

Task 1a: Justification of Design Decisions

In Deliverable 1, getHourlyRate() was implemented as a concrete method in the abstract User class using a switch statement on userType. For Deliverable 2, this was refactored to an abstract method, requiring each subclass (Student, Faculty, Researcher, Guest) to provide its own implementation. This change better supports the Open/Closed Principle, the system is now open for extension (adding a new user type with a new rate) without modifying the base class. It also pairs cleanly with the Factory Method pattern introduced in D2, as the factory creates fully-formed polymorphic objects that carry their own rate logic.

We selected six design patterns to address specific design challenges and fully support all 10 requirements:

1. Singleton (AuthenticationService, BookingManager)
The Singleton pattern was applied to AuthenticationService and BookingManager because both classes manage shared, stateful resources that must be consistent across the entire application. AuthenticationService maintains the map of active user sessions; If multiple instances existed, a user logged in through one instance would appear logged out to another, causing security failures across Req 1, 2, and 8. BookingManager coordinates all reservation creation, modification, and cancellation logic. Multiple instances could cause race conditions where two bookings are created for the same equipment at the same time, violating Req 3 and 4. By enforcing a single instance via a private constructor and a static getInstance() method, we guarantee a single source of truth for both session management and booking state throughout the application lifecycle.

2. Factory Method (UserFactory)
The Factory Method pattern was applied through the UserFactory class, which is responsible for instantiating the correct User subclass based on a provided UserType enum value. Without this pattern, every part of the codebase that needed to create a user (e.g., the registration screen, the admin panel, CSV loading in UserDAO) would need to contain a switch statement that maps UserType to a constructor call, leading to duplicated creation logic scattered across the application. With UserFactory, user creation is centralized, and adding a new user type in the future only requires changes in one place. This pattern also pairs directly with the D2 refactoring of getHourlyRate() to abstract: the factory creates fully polymorphic User objects, and the calling code never needs to know which subclass was created, it simply calls getHourlyRate() and gets the correct rate for Req 3 through polymorphism.

3. Observer (Sensor → ManagerDashboardController)
The Observer pattern was applied to implement Req 5, where sensors track equipment usage and send updates to the system. Sensor acts as the Subject (Observable); It holds a list of registered observers and notifies them when its operational data changes (e.g., a new usage reading, an abnormal status flag). ManagerDashboardController implements the SensorObserver interface and registers itself to receive these updates. When the Manager clicks the "Refresh" button in the UI, the sensor simulation reads the latest data from sensors.csv, updates its internal state, and calls notifyObservers(), which triggers update(SensorData) on the dashboard controller. This decouples the sensor subsystem from

the UI entirely, the Sensor class has no knowledge of JavaFX or any controller, it simply calls the observer interface. This makes the monitoring feature easily extensible: a NotificationService observer could be added later to send email alerts on abnormal readings without changing the Sensor class at all.

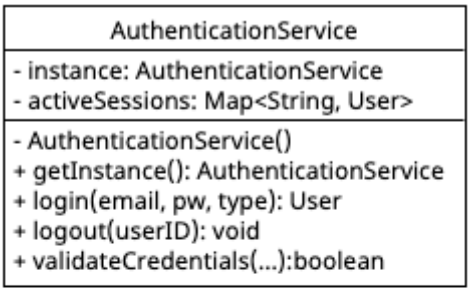
4. Strategy (PaymentService + payment method implementations)
The Strategy pattern was applied to implement Req 10, which requires the system to support multiple payment methods: credit card, debit card, institutional account, and research grant. Each payment method has distinct validation rules and processing logic, a research grant requires fund availability checking against ResearchGrant.availableFunds, while a credit/debit card requires card number and expiry validation. Without the Strategy pattern, PaymentService would contain a large if-else block that grows every time a new payment method is added. Instead, a PaymentStrategy interface defines the contract (processPayment, validatePayment), and four concrete classes implement it: CreditCardPayment, DebitCardPayment, InstitutionalAccountPayment, and ResearchGrantPayment. PaymentService holds a reference to whichever strategy is selected at runtime and delegates all payment processing to it. This satisfies the Open/Closed Principle: adding a new payment method only requires a new strategy class, with zero changes to PaymentService.

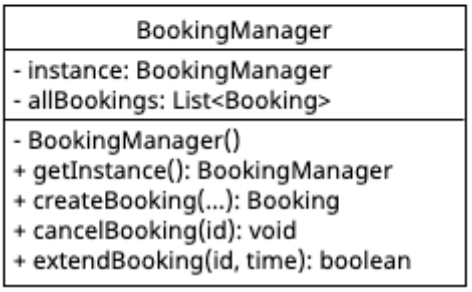
5. Facade (BookingFacade)
The Facade pattern was applied through the BookingFacade class, which provides a simplified interface to the complex booking subsystem. Creating a booking in this system requires coordinating at least six operations: verifying user credentials via AuthenticationService, checking equipment availability via EquipmentDAO, calculating the total cost and deposit using the user's hourly rate (Req 3, 4), processing the deposit payment via PaymentService, saving the booking record via BookingDAO, and sending a confirmation notification via NotificationService. Without a Facade, every UI controller that triggers a booking would need direct references to all six of these components and would need to orchestrate the full workflow itself, spreading complex business logic into the presentation layer. BookingFacade hides all of this behind three clean methods: createBooking(), cancelBooking(), and extendBooking(). The controllers only interact with the Facade, keeping the Presentation Layer thin and testable.

6. State (Booking status transitions)
The State pattern was applied to manage the lifecycle of a Booking object, which directly supports Req 8 (modify/cancel) and Req 9 (extend). A booking can exist in five states: PendingState, ConfirmedState, ActiveState, CompletedState, and CancelledState. Without the State pattern, the Booking class would contain a large series of if-statements checking this.status before every operation. For example, cancel() would need to check whether the booking is already active or completed before deciding what to do. The State pattern moves each behaviour into its own state class: ConfirmedState.cancel() processes a refund and transitions to CancelledState, while ActiveState.cancel() applies a cancellation penalty instead. Booking holds a reference to its current state and delegates all method calls to it. This ensures that invalid transitions (e.g., extending a CancelledState booking) are handled cleanly within the state class itself rather than through scattered conditional logic, making the codebase significantly easier to maintain and extend.

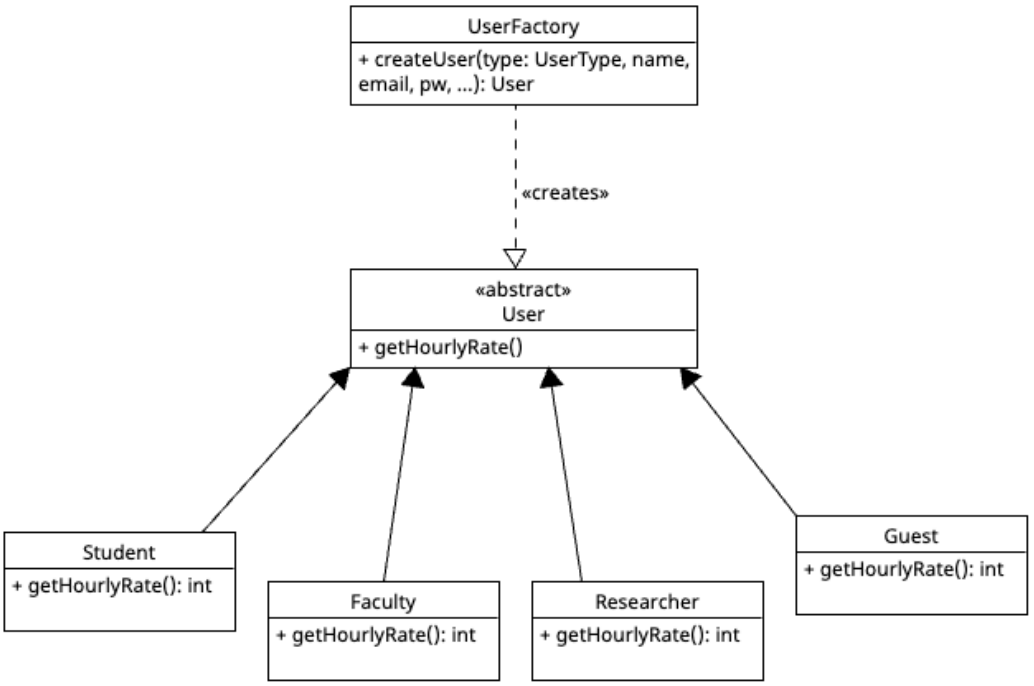
Task 1b: Class diagrams (contains six design patterns), you can use sub-diagrams to illustrate each of the six design patterns.

Pattern 1 - Singleton
Applied to: `AuthenticationService`, `BookingManager`

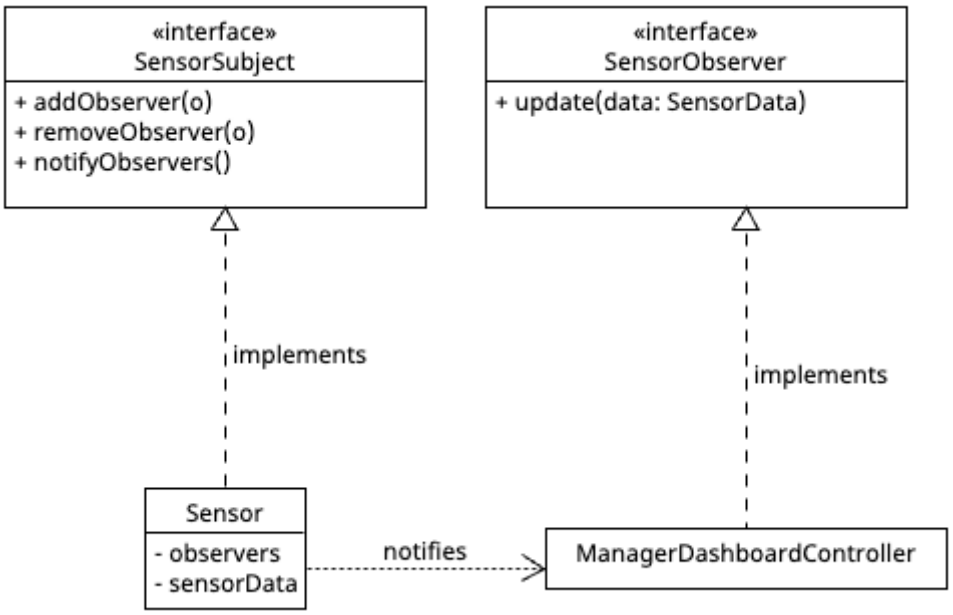




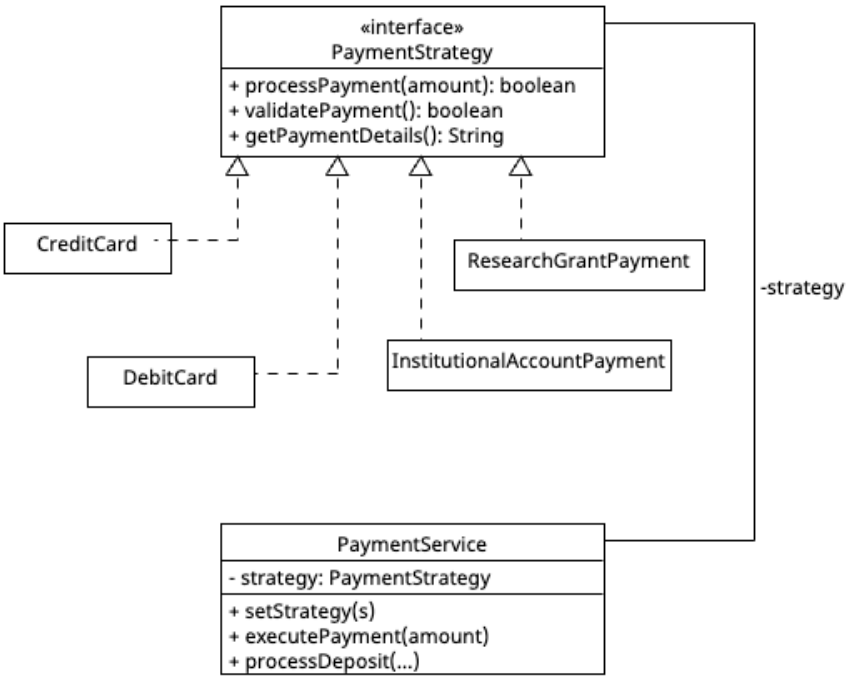
Pattern 2 - Factory Method
Applied to: `UserFactory`



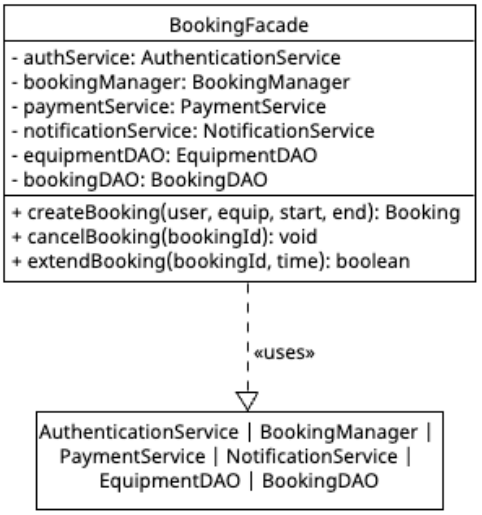
Pattern 3 - Observer
Applied to: `Sensor` (Subject) → `ManagerDashboardController` (Observer)



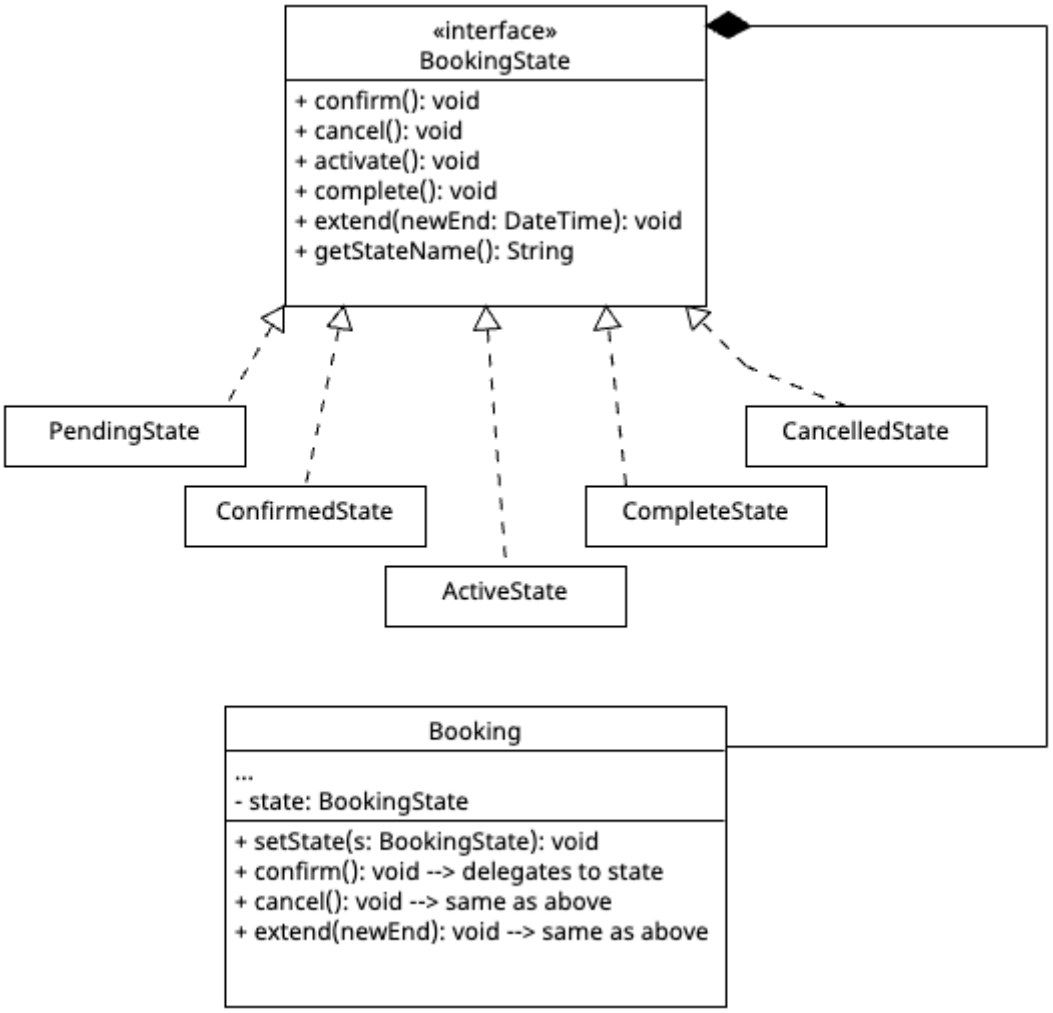
Pattern 4 - Strategy
Applied to: `PaymentService` + payment method implementations



Pattern 5 - Facade
Applied to: `BookingFacade`



Pattern 6 - State
Applied to: `Booking` status transitions

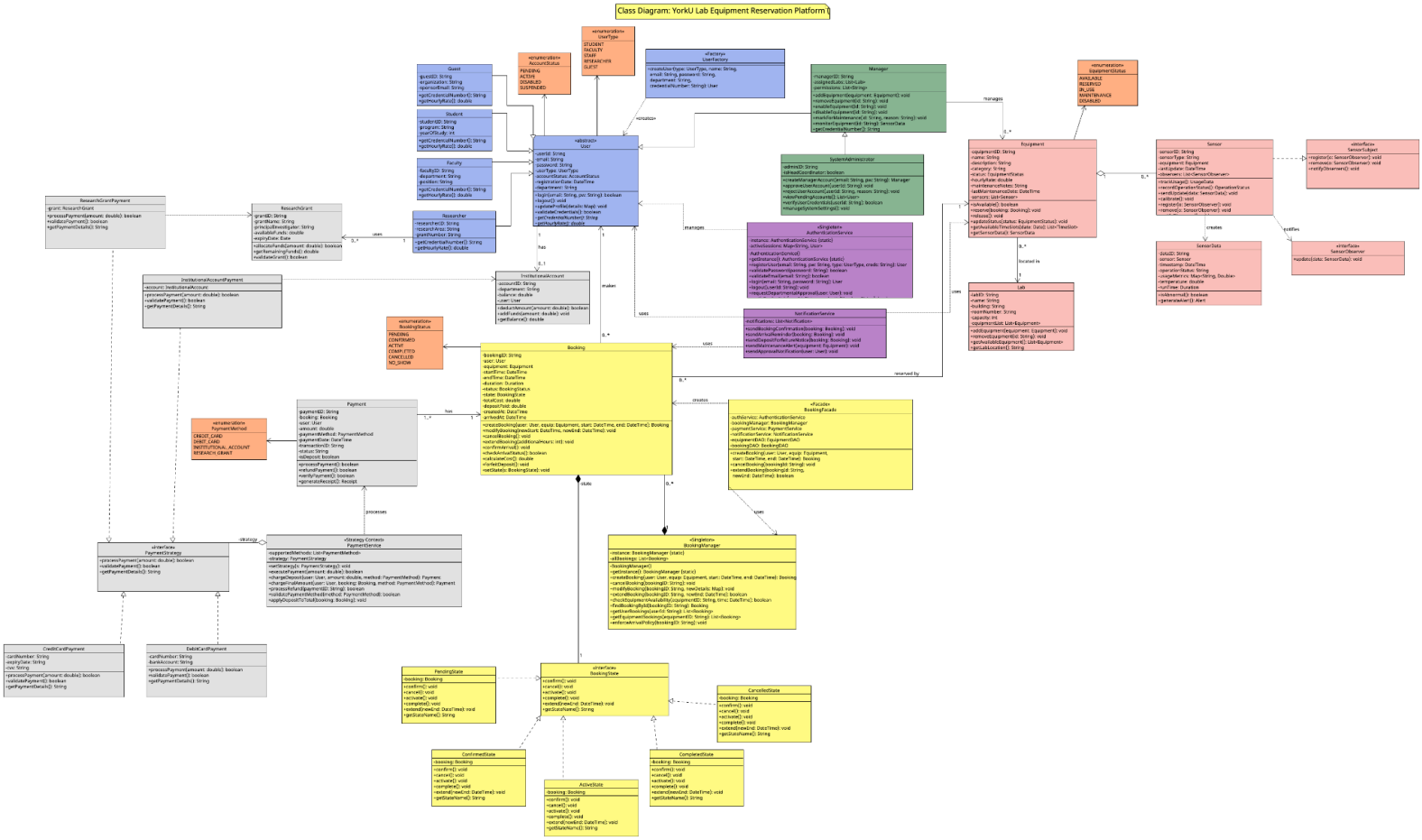


Every Single Change in the Class Diagram and Reasoning

Section	Element	Action	Reason
Booking	`Booking`	ADD `- state: BookingState` field	State pattern
Booking	`Booking`	ADD `+ setState()` method	State pattern
Booking	`BookingManager`	ADD `- instance`, private constructor, `+ getInstance()`	Singleton
Booking	`BookingState`	ADD new interface (6 methods)	State pattern
Booking	`PendingState`	ADD new class	State pattern
Booking	`ConfirmedState`	ADD new class	State pattern
Booking	`ActiveState`	ADD new class	State pattern
Booking	`CompletedState`	ADD new class	State pattern
Booking	`CancelledState`	ADD new class	State pattern
Booking	`BookingFacade`	ADD new class (3 methods, 6 fields)	Facade pattern
Payment	`PaymentService`	ADD `- strategy` field, `setStrategy()`, `executePayment()`	Strategy pattern
Payment	`PaymentStrategy`	ADD new interface (3 methods)	Strategy pattern
Payment	`CreditCardPayment`	ADD new class	Strategy pattern
Payment	`DebitCardPayment`	ADD new class	Strategy pattern
Payment	`InstitutionalAccountPayment`	ADD new class	Strategy pattern

Payment	`ResearchGrantPayment`	ADD new class	Strategy pattern
Equipment	`Sensor`	ADD `- observers` field + `register()`, `remove()`, `notifyObservers()`	Observer pattern
Equipment	`SensorSubject`	ADD new interface (3 methods)	Observer pattern
Equipment	`SensorObserver`	ADD new interface (1 method)	Observer pattern
User	`User`	MODIFY `getHourlyRate()` to (abstract)	Factory + polymorphism
User	`AuthenticationService`	ADD `- instance`, private constructor, `+ getInstance()`	Singleton
User	`UserFactory`	ADD new class	Factory Method pattern
Subclasses	`Student`, `Faculty`, `Researcher`, `Guest`	KEEP `getHourlyRate()` - now formally implements abstract	Factory + polymorphism

New Updated Class Diagram



The class diagram PNG can be accessed from google drive:
https://drive.google.com/drive/folders/1gWMrdNrTLJF9ljm9-rkeiyR5jx85HCTI?usp=sharing

EECS3311

Task 1c: Discussion about how each requirement can be achieved

Our system fully covers all 10 requirements through the layered architecture and six design patterns. Below, we map each requirement to the specific components, classes, and patterns responsible, followed by a summary table of pattern coverage.

Req1: User registration with role-specific approval

- Registration occurs via LoginController and UserDashboardController, creating users through UserFactory (Factory Method) based on UserType enum.[file:2] UserDAO persists to users.csv with AccountStatus.PENDING for Faculty/Researcher; AuthenticationService (Singleton) blocks login until Admin approval changes it to ACTIVE.

Req2: Secure login with role-based dashboards

- AuthenticationService.login() validates credentials/sessions (Singleton ensures single session state) and routes to role-specific dashboards (UserDashboardController, ManagerDashboardController, AdminDashboardController). SystemAdministrator is protected from public creation.

Req3: Booking fees by user type

- BookingFacade.createBooking() delegates to BookingManager (Singleton), which uses polymorphic getHourlyRate() on User subclasses (Student/Faculty/Researcher/Guest) + Equipment.hourlyRate to compute totalCost/deposit. Persisted via BookingDAO to bookings.csv.

Req4: Deposit payment, arrival window, no-show handling

- Deposits are processed via PaymentService (Strategy) before PendingState → ConfirmedState transition; BookingState enforces a 20-minute arrival window, and auto-cancels no-shows (retaining deposit in payments.csv).

Req5: Sensor monitoring and alerts

- Sensor (implements SensorSubject, Observer) reads sensors.csv, notifies registered SensorObserver (ManagerDashboardController) on abnormalities (temp>80°C, runtime>72h); updates Equipment status to UNDER_MAINTENANCE via EquipmentDAO.

Req6: Equipment add/disable/remove

- ManagerDashboardController + EquipmentDAO handle CRUD on equipment.csv; BookingManager blocks bookings on DISABLED/UNDER_MAINTENANCE equipment.

Req7: Lab/location management

- Equipment.labID links to Lab; managers select labs (Lassonde/Bergeron) during creation; filtered views in ManagerDashboardController.

Req8: Role-specific ID/certification + booking modify

- Registration UI shows dynamic fields (Student ID, Staff ID, etc.), stored in users.csv and displayed in dashboards; pre-start modifications via BookingFacade → BookingManager → current BookingState.validateModify().

Req9: Custom time slots + extension

- Users select start/duration; BookingManager checks overlaps in bookings.csv; extendBooking() validated by ActiveState → new endTime, persisted if no conflicts.

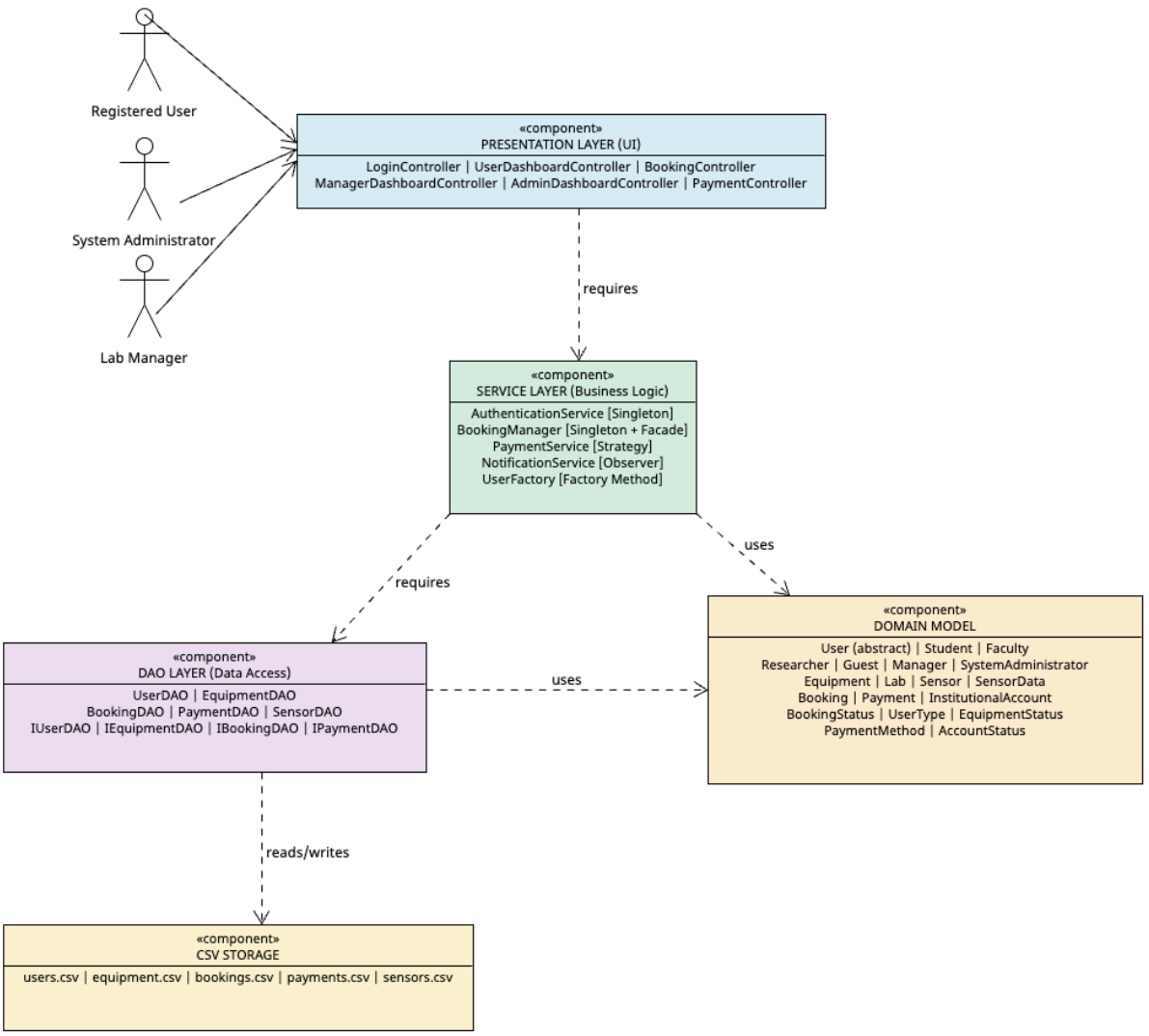
Req10: Payment methods with validation

- UI selects method → PaymentService.setStrategy() (CreditCardPayment/DebitCardPayment/InstitutionalAccountPayment/ResearchGrantPayment); each validates (e.g., grant balance check) before PaymentDAO write.

Patterns Implemented and what requirements it fulfills

#	Pattern	Applied To	Requirement(s)
1	Singleton	`AuthenticationService`, `BookingManager`	Req 1, 2, 3, 4, 8
2	Factory Method	`UserFactory` + abstract `getHourlyRate()`	Req 1, 3
3	Observer	`Sensor` → `ManagerDashboardController`	Req 5
4	Strategy	`PaymentService` + 4 payment implementations	Req 10
5	Facade	`BookingFacade`	Req 3, 4, 8, 9
6	State	`Booking` + 5 state classes	Req 8, 9

Task 1d: Component Diagram



Task 1e: Justification for component decomposition and interactions

Component	Provided Interfaces	Required Interfaces
Presentation (UI)	`<<UI>>` to actors	`IAuthService`, `IBookingService`, `IEquipmentService`, `IPaymentService`
Service Layer	`IAuthService`, `IBookingService`, `IEquipmentService`, `IPaymentService`, `INotificationService`	`IUserDAO`, `IEquipmentDAO`, `IBookingDAO`, `IPaymentDAO`
DAO Layer	`IUserDAO`, `IEquipmentDAO`, `IBookingDAO`, `IPaymentDAO`	`ICSVStorage`
CSV Storage	`ICSVStorage`	*(none - bottom of stack)*
Domain Model	`IDomain` (model objects)	*(none - pure data/logic)*

The YorkU Lab Equipment Reservation Platform is decomposed into five distinct components following a layered architectural style: the Presentation Layer (UI), the Service Layer (Business Logic), the Domain Model, the DAO Layer (Data Access), and the CSV Storage Layer. The Presentation Layer has every JavaFX controller and FXML view, and is the only part that the three system actors, Registered Users, Lab Managers, and System Administrators, work with directly. It relies only on the interfaces the Service Layer shows, so the UI doesn't know how data is saved or received. This makes sure the principle of separation of concerns is followed: each part has one clear job and speaks to the others through the interfaces it makes available.

The Service Layer handles the business logic and contains the main classes from the D1 class diagram: `AuthenticationService`, `BookingManager`, `PaymentService`, `NotificationService`, and `UserFactory`. All six of the design patterns used are in this layer. AuthenticationService and BookingManager are Singletons to have control in one place, UserFactory uses Factory to make user creation simple, and PaymentService uses Strategy to deal with the different ways to pay in Req 10. Under the Service Layer is the DAO Layer, which puts all CSV read and write actions into type-safe interfaces: IUserDAO, IEquipmentDAO, and so on; so that, if the way data is kept was changed from CSV files to a real database, only the DAO part would need to be changed. The Domain Model part is kept on its own, with no links out, and has all the things from the D1 class diagram, the User hierarchy, Equipment, Booking, Payment, Sensor, and all enums. This is used by the Service and DAO layers, making certain that domain logic is kept clean and can be used again.

This structure is a direct match for the system's needs as AuthenticationService handles Req 1-2, BookingManager does Req 3-4 and 8-9, the Sensor part of the Domain Model, which the Observer pattern in NotificationService tells, does Req 5, EquipmentDAO and the Manager service do Req 6-7, and PaymentService handles Req 10.

Task 2a: All Requirements Covered

The implemented JavaFX application successfully executes the logic detailed in Task 1c. All requirements (Req 1 through Req 10) are fully functional, verifiable via the application's GUI, and persist their state correctly to the mock database CSV files.

Task 2b: All Six Design Patterns Implemented

The system incorporates multiple design patterns to improve modularity, maintainability, and scalability. These patterns are applied across different components of the system to handle object creation, state transitions, system communication, and abstraction.

- **The State Pattern** is used in the Booking class to manage the booking lifecycle. Each booking transitions through states such as Pending, Confirmed, Active, Completed, and Cancelled. Instead of using conditional logic, each state encapsulates its own behaviour, ensuring that only valid transitions are allowed and improving code readability and extensibility.
- **The Observer Pattern** is used for sensor tracking. Each Sensor acts as a subject, while the backend system acts as an observer. When abnormal sensor data is detected, all registered observers are notified automatically. This allows the system to react in real time by updating equipment status and cancelling affected bookings without tight coupling between components.
- **The Singleton Pattern** is applied in the BookingManager class to ensure that only one instance of the manager exists throughout the application. This centralizes booking logic and sensor handling, ensuring consistent behaviour across the system.
- **The Factory Pattern** is implemented through the UserFactory class, which creates different types of users (Student, Faculty, Researcher, etc.) based on input parameters. This abstracts object creation logic and makes it easier to extend the system with new user types.
- **The Facade Pattern** is used in the BookingFacade class to provide a simplified interface for controllers to interact with complex booking operations. Instead of directly interacting with multiple classes such as DAO, Booking, and Equipment, the controller communicates with the facade, improving separation of concerns.
- **The Data Access Object (DAO) Pattern** is used for persistence. Classes such as UserDAO, BookingDAO, and EquipmentDAO handle all interactions with CSV files, separating data storage logic from business logic. This makes the system easier to maintain and allows future replacement of CSV storage with a database without affecting the core system.

Task 3: Demo video

Video can be found in:

https://drive.google.com/drive/folders/1gWMrdNrTLJF9Ijm9-rkeiyR5jx85HCTI?usp=sharing

Or on youtube:

https://youtu.be/AEp4SC17Ih4

Task 4. AI-Assistant Adaptation Report

a. Role of AI in tasks 1 and 2

During the redesign and implementation of our Lab Equipment Reservation Platform, we utilized Google's Gemini 3.1 Pro model as our primary AI assistant. The AI was used extensively across design pattern selection, UML architectural planning, and JavaFX implementation.

Prompts Provided: We provided the AI with our project requirements (Req 1-10), existing code snippets, and structural questions such as, "What design patterns fit these requirements?", "This is my current initialize method... shouldn't I change this in every dashboard controller?", and "How should we implement the usage of sensors to our program?"

Outputs Generated: The AI generated UML-aligned architectural suggestions (recommending the Facade, Strategy, State, Observer, Singleton, and Factory Method patterns). It also provided step-by-step implementation guides, JavaFX boilerplate code, formatting logic for our table columns, and mock CSV data to simulate our database.

Interpretation and Application: Outputs were rarely accepted completely "as-is" without review. While we accepted the high-level structural pattern suggestions, we frequently modified the generated Java code to fit our specific Data Access Object (DAO) CSV logic. For instance, we applied the AI's suggestion to use TableCell factories for formatting LocalDateTime strings, but we actively adapted it across our Admin, Manager, and User controllers to match our specific variable names.

b. Arguments and resolution

There were two notable instances where our understanding of the system design differed from the AI's assumptions, requiring intervention and resolution:

Instance 1. The Booking Confirmation Workflow (Req 4): Early in the implementation, the system required a Lab Manager to manually click "Approve" for every pending booking. We questioned the AI on whether this was the intended way to design an automated system. The AI analyzed the project phases and agreed that the manual flow was clunky and incorrect.

Resolution: We collaboratively pivoted the design. We removed the manual manager bottleneck and instead tied the PENDING to CONFIRMED state transition strictly to an automated upfront deposit payment.

Instance 2. Sensor Implementation (Req 5): The AI initially interpreted the "sensor" requirement through a strict hardware lens, suggesting we build architecture to monitor physical temperature spikes (e.g., > 80°C). We recognized a technical issue: our project is purely software-based with no physical sensors, making this interpretation overly complex and disconnected from the simulation environment.
Resolution: We explicitly corrected the AI, asking it to rethink the requirement "as if no files had been written for it." The AI adjusted its reasoning and suggested a "Wear and Tear" software tracker. We accepted this resolution, implementing a system that calculates historical usage duration from the CSV database and trips a maintenance flag when an equipment piece surpasses 72 hours of total use.

c. Verification of AI responses
We did not blindly trust the AI's code; every suggestion was verified against Deliverable 2 UML standards, our rubric (Req 1-10), and Java compilation rules.

Verification Process: We manually traced the AI's suggested state transitions to ensure they adhered to the State Pattern rules (e.g., verifying that users cannot pay balances on PENDING or CANCELLED bookings). We also ran rigorous UI testing to ensure the JavaFX views matched the backend controllers.

Concrete Example 1 (Rejected/Modified - Unreachable Code): During the implementation of the dynamic booking fees (Req 3), AI-assisted code was drafted to calculate base costs and apply a flat fee via a switch statement based on UserType. Upon code review, we identified a critical Java compilation error: the code contained a premature return statement before the switch block, making the fee calculation entirely unreachable.
Resolution: We modified the code by deleting the early return, ensuring the control flow reached the Strategy-like fee application.

Concrete Example 2 (Modified - Memory vs. CSV Mismatch): The AI provided a "Simulate Sensor Failure" UI button to demonstrate the Observer pattern for Req 5. However, upon testing, we verified that the application threw an error because the Equipment objects newly parsed from the CSV lacked instantiated Sensor objects in memory.
Resolution: We modified the AI's architecture by introducing a "Lazy Initialization" pattern. We updated the backend to dynamically instantiate and register sensors on the fly if an equipment piece was loaded from the database without one, perfectly resolving the crash and fulfilling the requirement.